ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING
DEPARTMENT

CENG 407

Conclusion & Future Work

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Overview

This document brings together the work presented across the preceding ones and reflects on what the TARAS project has achieved as a whole. TARAS (Tarım Analizi Sistemi) was developed as an end-to-end precision-agriculture decision-support platform that integrates IoT-based field sensing, physics-grounded irrigation analysis, computer-vision-based disease detection, an LLM-driven advisory layer, and a mobile application built around a digital-twin visualization of the field. The intent was never to automate farming, but to give farmers a single, accessible interface that turns raw sensor and image data into clear, explainable guidance. The conclusions below summarize what was built, where it currently stands, and the limitations that shaped both the design decisions and the direction of future work.

The project began with a broad survey of smart-agriculture features and was deliberately narrowed, based on impact, feasibility, cost, and sustainability, to the components that could be delivered to a defensible standard within the project timeline. What follows is a synthesis of the outcomes from each subsystem, an honest account of the limitations that remain, and a structured plan for the work that should come next.

2. Summary of Contributions

2.1. Hardware and Field Sensing

The hardware layer was designed around an ESP32-C6 sensor node paired with a mains-powered ESP32 WROOM-32 gateway. Each sensor node samples soil moisture through a capacitive probe and temperature and humidity through an SHT31 over I²C, running on a 3.7 V Li-Po battery, deep-sleep cycling, and a per-unit calibration step that absorbs probe-to-probe variation. The node-to-gateway link uses ESP-NOW with HMAC signing and replay protection, while the gateway-to-backend channel was migrated to HTTPS:443 with full certificate-chain validation against the Let's Encrypt root CA. Across 52 defined hardware test cases, 42 passed, 3 were partial, and 7 remain pending; nothing reached a FAIL state. The deployed gateway has been running continuously on the final firmware since the upgrade with no observed regressions, which serves as a second source of evidence alongside the targeted bench and integration tests.

2.2. Backend, Cloud, and Data Layer

The backend was implemented as a Node.js API service on AWS, fronted by a PostgreSQL database that holds users, farms, fields, zones, plantings, time-series telemetry, irrigation recommendations, carbon-footprint records, and disease diagnoses. Disease inference was deployed as a Lambda function backed by a three-model classifier ensemble (a ConvNeXt-V2-Base, a Swin-Small, and a field-tuned Swin-Base), including a queue-and-resilience layer that survives transient failures by reverting requests to a QUEUED state and retrying through a cron-driven scheduler. Cloud-side optimization work brought the disease-inference cold-start down from roughly sixty seconds to approximately fourteen seconds across multiple iterations, with subsequent effort redirected toward cost per invocation rather than further latency reduction. The result is a backend that behaves well under intermittent connectivity, retries gracefully when downstream services are unavailable, and exposes a clean REST surface for the mobile client.

2.3. Irrigation Recommendation Module

The irrigation recommendation module was implemented as a decision-support feature that uses real-time soil moisture readings, crop information, field configuration and planting data to determine whether irrigation is needed. Instead of following a fixed irrigation schedule, the system evaluates the current condition of each zone and generates recommendations such as whether the crop should be irrigated, the suggested irrigation time, urgency level, target soil moisture and estimated water amount.

Each recommendation is stored in the database as an irrigation job, allowing the system to track pending, executed, skipped, and analyzed irrigation decisions. After irrigation is performed, follow-up sensor readings can be used to compare the predicted soil moisture response with the actual result. This creates a feedback loop that supports calibration and improves the reliability of future recommendations.

The main contribution of this module is that it turns raw sensor data into practical, zone-specific irrigation guidance. By helping farmers avoid both over-irrigation and under-irrigation, the module supports water efficiency, crop health, and more informed field management.

2.4. Disease Detection Module

The vision-based classifier converged on a revised fourteen-class label schema covering the most relevant disease and healthy categories across the supported crops. A three-model ensemble was selected as the final deployment configuration after a controlled benchmark in which a pool of backbones, including ConvNeXt-V2-Base, Swin-Small, Swin-Tiny, and Swin-Base at both 224 px and 384 px, was trained under identical conditions on the same revised dataset, and the best-performing subset was combined. On the held-out test set (1,941 images), the deployed ensemble reached an overall accuracy of 0.9701 and a macro F1 of 0.9697. The more informative figure is the real-world field slice (332 images), where the macro F1 was 0.8488, compared with 0.9944 on the cleaner laboratory (PlantVillage-style) slice (1,594 images). This field-slice result is notable because it was measured under deployment-oriented conditions rather than on a clean PlantVillage-style benchmark, which makes it a stronger predictor of behaviour in actual field use.

The most consequential lesson from this subsystem was that the largest improvements did not come from chasing a stronger backbone, but from aligning the label schema, the dataset composition, and the evaluation protocol with the conditions the model would actually encounter. The composite validation score (0.5 × overall val macro F1 + 0.5 × field-slice macro F1) operationalized this principle during model selection, and the project treats the field (real-world) slice as the primary indicator of deployment readiness.

2.5. LLM Advisory Module

The advisory layer was built as an agentic tool-loop running on top of an LLM with a small, scoped set of read-only tools and a curated knowledge base of around 146 fragments across eight categories. Its role is to translate the structured outputs of the rest of the system, sensor readings, irrigation recommendations, disease predictions, and field metadata, into context-aware natural-language guidance that the farmer can act on. Provider fallback, iteration limits, result-size caps, scope checks, and audit logging were added to keep the agent grounded and to bound the risks introduced by autonomous tool selection. End-to-end response times in production typically fall between three and six seconds. Evaluation of this module remains qualitative, based on coherence, grounding, and consistency with upstream system outputs, since a large-scale quantitative study would require a labelled corpus that does not yet exist for this specific task.

2.6. Mobile Application and Digital Twin

The mobile application was implemented in React Native with Expo, chosen for cross-platform deployment and for the iteration speed it permits on a small team. Its main surfaces are the dashboard with sensor readings and a 2D/3D moisture visualization, the disease-detection flow with camera capture and history-folder organization, the irrigation timetable, the carbon-footprint logger, and a global advisory assistant overlay that exposes the LLM module to the user. The digital twin uses low-poly geometry (rendered with three.js / React-Three-Fiber) and an SVG overlay strategy over a standard mobile map so that spatial moisture visualization runs smoothly on a standard smartphone, rather than requiring the desktop-class hardware that high-end agronomic twins typically assume. Throughout the project, the design defaulted to decision support rather than full automation: every recommendation is presented in a form the farmer can override, and the AI assistant is consistently framed as a guide rather than as an authority.

2.7. Sustainability and Carbon Footprint

Carbon-footprint estimation was implemented as a user-facing feature that allows farmers to log fuel, electricity, and fertilizer usage. Each input is converted into an estimated CO_2 equivalent value using the standard emission-factor approach:

$$\textbf{CO}_2\textbf{e Emission = Activity Amount} \times \textbf{Emission Factor}$$

Here, the activity amount represents the user entered value, such as liters of fuel, kilowatt-hours of electricity, or kilograms of fertilizer. The emission factors were selected from commonly used international references such as IPCC and UN-related sustainability guidelines. Each calculated result is saved as a carbon record, allowing the system to show total emissions and category-based emission history. This supports the project's connection with SDG 2, SDG 9, SDG 12, and SDG 13.

3. Reflections on the Design Philosophy

Three design commitments shaped the project from the literature-review stage onward, and they held up under implementation pressure. The first was decision support rather than automation: every analytical output is presented as a recommendation that the farmer can act on, ignore, or adjust, rather than as an instruction issued to an actuator. This kept the trust model simple and avoided the safety and accountability problems that come with closed-loop control of irrigation hardware. The second was real-world evaluation over laboratory-style benchmarks. The disease classifier in particular benefited from this commitment, since the strongest gains came from improving the realism of the evaluation pipeline rather than from architectural changes. The third was a mobile-first orientation, which was reflected in the choice of React Native, the low-poly digital twin, and the deliberate avoidance of features that would require desktop-class hardware in the field.

Each of these design commitments introduced certain trade-offs. Because TARAS is framed as a decision-support system rather than a fully automated control system, it cannot independently prevent a farmer from over-irrigating; instead, it provides evidence-based explanations that help the farmer understand why over-irrigation would be a poor decision. Similarly, evaluating the disease model on real-world test data resulted in lower headline accuracy than a PlantVillage-only benchmark might have produced, but these results offer a more realistic and honest indication of deployment performance. The mobile-first design also limited the use of more complex visualization techniques that would be easier to render on workstation-level hardware. These trade-offs were intentional, and preserving this balance between practicality, transparency, and deployability remains important as the system evolves.

4. Limitations

The limitations identified during development can be grouped into four main areas, each of which defines an important direction for future work.

First, there are limitations related to scope. The disease classifier is currently limited to fourteen categories across a small number of crops, and it does not yet include an out-of-distribution rejection class. As a result, images outside the supported scope, such as unseen diseases, nutrient deficiencies, pest damage, or abiotic stress symptoms, may still be forced into one of the known classes. At this stage, the backend confidence threshold is the main mechanism used to reduce unreliable predictions. Similarly, the LLM knowledge base covers common agricultural cases, but it is not encyclopedic. Questions outside its retrieval scope may therefore rely more heavily on the model's general knowledge.

Second, the evidence base is limited in terms of coverage and duration. Hardware reliability data currently spans hours to days rather than weeks or months, so long-term battery life and multi-week stability are still extrapolated from shorter tests instead of being measured directly. Outdoor testing has also been limited to controlled damp-exposure tests and bench conditions that approximate field use, rather than full diurnal cycles involving sun, rain, wind, and dust. In addition, enclosure ingress testing remains pending, and scale-related claims have not yet been validated through multi-gateway deployments with tens of sensors per gateway.

Third, connectivity remains a practical constraint. Both the LLM advisory module and the disease-detection Lambda require an internet connection to generate new outputs, which may be problematic in rural areas with unstable connectivity. Although the mobile client can display the last-known system state while offline, it cannot generate new advisory responses or perform new disease classifications without a connection.

Fourth, the advisory module still requires deeper evaluation. The LLM module has been demonstrated end-to-end in production, but it has not yet undergone a large-scale quantitative evaluation using labelled rubrics for groundedness, helpfulness, and safety. The current evidence base is therefore qualitative. A more formal evaluation will be necessary before the module can be presented as anything beyond a decision-support aid.

5. Future Work

The remaining work is largely incremental: the core architecture is stable, and what remains is to extend coverage, improve robustness, and close specific gaps identified during development. The items below are grouped by subsystem and ordered loosely by estimated impact on the user experience.

5.1. Hardware and Field Deployment

The most valuable next step on the hardware side is a real outdoor deployment of at least one gateway and several sensors for two weeks or longer across changing weather, with continuous diagnostic capture. This would convert the current short-run reliability evidence into observed long-term behaviour and would expose any failure modes that bench testing cannot reach. Alongside this, the partial and pending test rows should be closed out, including formal deep-sleep and active-current measurements with a precision current analyser, the formal enclosure ingress test, the remaining adversarial security cases, and the outstanding OTA fault-injection variants.

Beyond coverage, two scalability questions remain open. The first is multi-sensor scalability on a single gateway, which would surface any concurrency issues in ESP-NOW packet interleaving and backend ingestion. The second is synthetic backend load: replaying sensor traffic at multiples of the normal rate and measuring backend latency and database throughput, which is needed before any claim about farm-scale rather than test-field-scale deployment. An automated regression harness wrapping the main integration tests (HMAC, pairing, end-to-end, OTA) would turn firmware releases from one-off events into a routine.

5.2. Disease Detection Module

Future improvements should build on the revised fourteen-class system rather than returning to a broader but less reliable label set. The first priority is targeted dataset expansion for the hardest categories, particularly the corn and tomato leaf-spot classes (bacterial_spot, septoria_leaf_spot, corn_gray_leaf_spot, and early_blight) which remain the weakest on the real-world slice. Adding more real-world images from Turkish greenhouse and open-field conditions would likely produce a larger gain than switching to a stronger backbone.

The second priority is confidence calibration and explicit rejection handling. A calibrated threshold and an out-of-distribution rejection path would let the system respond with "low confidence, please capture another image or use the advisory module" rather than forcing a class. Visual explanation maps (for example, attention or Grad-CAM-style overlays) would also improve user trust by showing which regions of the leaf drove the prediction. Further architecture experiments should be selective: the current results already identify the selected three-model ensemble (with Swin-Small as its strongest single member) as a strong deployment configuration, and the next round of work should improve this pipeline rather than restart the comparison.

5.3. Irrigation Recommendation Module

The irrigation recommendation module should be improved through longer-term field validation, stronger calibration, and better use of environmental context. Although the system can currently generate zone-specific irrigation jobs using soil-moisture readings, crop information, field configuration, and planting data, its reliability still needs to be tested across different soil types, crop stages, and outdoor conditions.

A key next step is to collect more post-irrigation follow-up data. By comparing predicted soil-moisture changes with actual sensor readings after irrigation, the system can refine its calibration parameters and reduce prediction error over time. Future versions should also incorporate weather-aware reasoning, including rainfall forecasts, temperature, humidity, and evapotranspiration estimates, so that recommendations can adapt to changing environmental conditions.

User feedback should also be integrated into the irrigation history. Whether farmers accept, modify, delay, or ignore a recommendation can provide useful evidence for improving personalization and practical usability. Finally, a controlled pilot study comparing

TARAS-assisted irrigation with unassisted irrigation would help measure real outcomes such as water savings, soil-moisture stability, crop-health effects, and user trust.

5.4. LLM Advisory Module

Four improvements stand out. First, the curated knowledge base should be expanded to cover unusual pest-disease interactions, region-specific crop guidance, and less common soil types, while remaining curated and attributable rather than scraped from the open web. Second, the tool set should be extended with a weather-forecast tool, a historical-analogue tool for comparing current conditions with similar past cases, and a push-notification tool for proactive alerts. Third, offline and low-bandwidth support should be considered: a small on-device fallback model could answer a narrow set of common questions using cached field data such as irrigation status, recent disease results, and basic crop-stage guidance. Fourth, the module needs a feedback loop. A thumbs-up/thumbs-down mechanism on advisory responses would identify weak answers, inform system-prompt revisions, and guide knowledge-base updates. Additional improvements that fit naturally into this roadmap include conversation summarization, voice input, what-if irrigation scenarios, and finer-grained per-tool authorization.

5.5. Platform and User Experience

On the platform side, the next round of work should consolidate features that already exist into a more cohesive user experience. This includes deeper integration between the carbon-footprint tracker and the irrigation timetable so that water and energy savings are surfaced in the same view as the recommendations that produced them, a stakeholder-facing dashboard that aggregates anonymized field outcomes for agricultural cooperatives and advisory bodies, and improved offline behaviour in the mobile client so that the digital twin and the most recent recommendations remain useful even when connectivity is intermittent. Internationalization beyond Turkish, particularly for languages spoken in agriculturally significant regions, would also broaden the platform's reach.

5.6. Longer-term Directions

Looking further ahead, the natural extensions of the current platform are multi-image and time-series reasoning for disease progression rather than single-leaf classification, tighter coupling between weather forecasts and the irrigation engine so that recommendations explicitly account for predicted rainfall and evapotranspiration windows, and a controlled pilot study with a small number of cooperating farms to measure real water and energy savings against an unassisted baseline. A pilot of this kind would also generate the labelled production data that the advisory module would need for a more rigorous quantitative evaluation, closing the loop between deployment and continued improvement.

6. Closing Remarks

TARAS set out to combine sensing, prediction, visualization, and advisory functions into a single mobile-first platform that small and mid-scale farmers could actually use, and to do so without overstating what any individual component can deliver. The hardware is deployed, the backend is in production, the disease classifier reaches roughly 97 percent overall accuracy on the held-out test set, with about 0.85 macro F1 on the real-world field slice, the advisory agent runs end-to-end with grounded responses in a few seconds, and the digital twin renders smoothly on commodity smartphones. Each of these components has limitations, and those limitations have been documented honestly rather than minimized.

What makes the project worth continuing is precisely the fact that the remaining gaps are addressable. Real outdoor deployments, dataset expansion in the harder disease categories, knowledge-base growth and feedback loops in the advisory layer, offline fallbacks, and a controlled pilot study would together move the platform from a working prototype to a system whose value can be measured in saved water, saved energy, and earlier disease intervention. The architecture, the design philosophy, and the evidence accumulated over the course of this project all point in the same direction: the next steps are clear, they are incremental, and they are within reach.